MedBook -- Modello fisico della base di dati MySQL

1. Tabella UTENTE

Campo	Tipo	NULL	Vincoli
`id_utente`	`INT UNSIGNED`	NO	PK, AUTO_INCREMENT
`nome`	`VARCHAR(100)`	NO	
`cognome`	`VARCHAR(100)`	NO	
`email`	`VARCHAR(255)`	NO	UNIQUE
`password_hash`	`VARCHAR(255)`	NO	
`ruolo_gestionale`	`VARCHAR(20)`	SÌ	CHECK
`stato_accesso`	`VARCHAR(20)`	NO	DEFAULT `ATTIVO` , CHECK
`generazione_autenticazione`	`BIGINT UNSIGNED`	NO	DEFAULT 0
`modificato_da`	`INT UNSIGNED`	SÌ	FK → `utente.id_utente`
`modificato_il`	`DATETIME`	SÌ	

`utente` contiene i dati comuni a tutti gli account e le informazioni necessarie all'autenticazione.

La password non viene memorizzata in chiaro: `password_hash` contiene il risultato della funzione di hashing adottata dal backend.

`ruolo_gestionale` può assumere:

```
AMMINISTRATORE
OPERATORE_DESK
NULL
```

L'appartenenza a `medico` o `paziente` non è invece rappresentata tramite questo campo.

`stato_accesso` ammette `ATTIVO` e `DISABILITATO` ; il relativo CHECK utilizza la collation `utf8mb4_0900_bin` . `generazione_autenticazione`
è un contatore associato alla gestione dell'autenticazione.
`modificato_da` è una FK autoreferenziale verso `utente.id_utente` , con `ON UPDATE RESTRICT` e `ON DELETE SET NULL` . `modificato_il` registra
la data e l'ora della modifica.

La regola secondo cui un medico non può contemporaneamente avere un ruolo gestionale richiede l'analisi di più tabelle ed è pertanto demandata al service Java.

2. Tabella MEDICO

Campo	Tipo	NULL	Vincoli
`id_utente`	`INT UNSIGNED`	NO	PK, FK → `utente.id_utente`
`specializzazione`	`VARCHAR(150)`	NO	
`numero_iscrizione_ordine`	`VARCHAR(50)`	NO	UNIQUE
`stato`	`VARCHAR(20)`	NO	DEFAULT `ATTIVO` , CHECK

`medico` realizza fisicamente una delle specializzazioni di `utente` .
Il campo `stato` ammette `ATTIVO` e `INATTIVO` e descrive lo stato del profilo medico, distinto dallo stato di accesso dell'account.

`id_utente` è contemporaneamente PK e FK. Non viene introdotto un identificativo separato del medico.

La FK verso `utente` utilizza `ON DELETE CASCADE` : eliminando il supertipo non avrebbe senso conservare la relativa specializzazione.

3. Tabella PAZIENTE

Campo	Tipo	NULL	Vincoli
`id_utente`	`INT UNSIGNED`	NO	PK, FK → `utente.id_utente`
`codice_fiscale`	`CHAR(16)`	NO	UNIQUE
`data_nascita`	`DATE`	NO	
`telefono`	`VARCHAR(20)`	SÌ	

Anche `paziente` realizza una specializzazione di `utente`.

Il codice fiscale viene rappresentato mediante `CHAR(16)`, mentre il telefono è testuale per poter conservare prefissi, simbolo `+` e zeri iniziali.

Lo stesso `id_utente` può essere presente contemporaneamente in `medico` e `paziente`.

4. Tabella PRESTAZIONE

Campo	Tipo	NULL	Vincoli
`id_prestazione`	`INT UNSIGNED`	NO	PK, AUTO_INCREMENT
`nome`	`VARCHAR(150)`	NO	
`descrizione`	`TEXT`	SÌ	
`durata`	`SMALLINT UNSIGNED`	NO	CHECK 1..60
`prezzo`	`DECIMAL(10,2)`	NO	CHECK ≥ 0

`prestazione` rappresenta il catalogo comune.

Nel modello fisico `durata` viene interpretata come **numero di minuti**. Nella versione corrente di MedBook sono ammesse prestazioni con durata compresa tra 1 e 60 minuti. Una prestazione di durata inferiore a 60 minuti occupa comunque l'intero slot standard di 60 minuti; prestazioni di durata superiore non sono gestite. Il prezzo è memorizzato con precisione decimale.

Il nome non viene utilizzato come identificatore: l'identità della prestazione è determinata da `id_prestazione` .

5. Tabella MEDICO_PRESTAZIONE

Campo	Tipo	NULL	Vincoli
`id_medico`	`INT UNSIGNED`	NO	PK, FK → `medico.id_utente`
`id_prestazione`	`INT UNSIGNED`	NO	PK, FK → `prestazione.id_prestazione`

La tabella associativa realizza la relazione molti-a-molti tra medici e prestazioni.

La chiave primaria è composta:

```
(id_medico, id_prestazione)
```

e identifica le combinazioni medico-prestazione ammesse.

La cancellazione della specializzazione del medico elimina le relative associazioni; una prestazione già referenziata non viene invece eliminata a cascata.

6. Tabella STANZA

Campo	Tipo	NULL	Vincoli
`id_stanza`	`INT UNSIGNED`	NO	PK, AUTO_INCREMENT
`nome`	`VARCHAR(100)`	NO	UNIQUE
`descrizione`	`VARCHAR(255)`	SÌ	

`stanza` identifica le risorse fisiche del singolo poliambulatorio.

Il nome viene reso univoco per evitare due stanze con la medesima denominazione all'interno della struttura.

7. Tabella DISPONIBILITA

Campo	Tipo	NULL	Vincoli
`id_disponibilita`	`INT UNSIGNED`	NO	PK, AUTO_INCREMENT
`data_ora_inizio`	`DATETIME`	NO	
`data_ora_fine`	`DATETIME`	NO	CHECK fine > inizio
`id_medico`	`INT UNSIGNED`	NO	FK → `medico.id_utente` ; parte di FK composta
`id_prestazione`	`INT UNSIGNED`	SÌ	parte di FK composta
`id_stanza`	`INT UNSIGNED`	SÌ	FK → `stanza.id_stanza`

La coppia:

```
(id_medico, id_prestazione)
```

costituisce una FK composta verso:

```
medico_prestazione(id_medico, id_prestazione)
```

Questa scelta impedisce, quando `id_prestazione` è valorizzato, di configurare uno slot per una combinazione medico-prestazione non autorizzata.

La disponibilità viene gestita in **due fasi**:

1. il Medico inserisce una fascia temporale; MedBook la suddivide in slot standard di **60 minuti**, valorizzando `id_medico` , `data_ora_inizio` e `data_ora_fine` ;
2. l'Operatore amministrativo / Segreteria configura ciascuno slot associando `id_prestazione` e `id_stanza` .

Per questo motivo `id_prestazione` e `id_stanza` sono fisicamente ammissibili a `NULL` . Uno slot è però **prenotabile soltanto quando entrambi sono valorizzati**.

Il database verifica che `data_ora_fine > data_ora_inizio` ; il `DisponibilitaService` genera esclusivamente slot di 60 minuti e impedisce:

- due slot sovrapposti dello stesso medico;
- quando la stanza è valorizzata, due slot sovrapposti nella stessa stanza;
- la configurazione con una prestazione non erogabile dal medico.

La scelta dell'idoneità funzionale della stanza resta responsabilità dell'Operatore amministrativo; il sistema ne controlla la disponibilità temporale.

Sono previsti indici su medico/intervallo e stanza/intervallo per supportare efficientemente queste verifiche.

8. Tabella PRENOTAZIONE

Campo	Tipo	NULL	Vincoli
`id_prenotazione`	`INT UNSIGNED`	NO	PK, AUTO_INCREMENT
`id_paziente`	`INT UNSIGNED`	NO	FK
`id_disponibilita`	`INT UNSIGNED`	NO	FK
`gestita_da`	`INT UNSIGNED`	SÌ	FK
`id_medico_effettivo`	`INT UNSIGNED`	SÌ	FK
`stato`	`VARCHAR(20)`	NO	DEFAULT `RICHIESTA` , CHECK
`indisponibilita_segnalata`	`BOOLEAN`	NO	DEFAULT FALSE, CHECK 0 o 1
`data_ora_richiesta`	`DATETIME`	NO	
`data_ora_gestione`	`DATETIME`	SÌ	
`prezzo`	`DECIMAL(10,2)`	NO	CHECK ≥ 0

Gli stati ammessi sono:

```
RICHIESTA
CONFERMATA
RIFIUTATA
ANNULLATA
EFFETTUATA
PAZIENTE_ASSENTE
NON_EROGATA
```

Ogni nuova prenotazione nasce nello stato `RICHIESTA`.

`indisponibilita_segnalata` registra la segnalazione di indisponibilità sulla singola prenotazione. Non rappresenta un periodo di assenza e non blocca automaticamente gli slot liberi. In MySQL `BOOLEAN` è un sinonimo di `TINYINT(1)`; il CHECK limita il valore a 0 o 1.

`gestita_da` e `data_ora_gestione` devono essere entrambi valorizzati oppure entrambi assenti.

`prezzo` rappresenta il prezzo storico della prenotazione. Il backend lo ricava dalla prestazione al momento della richiesta; non deve accettare come autorevole un prezzo inviato dal frontend.

Storico dello slot

Non viene imposto `UNIQUE(id_disponibilita)`, perché uno stesso slot può conservare più prenotazioni storiche rifiutate o annullate.

Deve tuttavia esistere **al massimo una prenotazione occupante per slot**. Sono considerati occupanti gli stati:

```
RICHIESTA
CONFERMATA
EFFETTUATA
PAZIENTE_ASSENTE
NON_EROGATA
```

`RIFIUTATA` e `ANNULLATA` liberano invece lo slot.

La verifica viene eseguita dal backend in una transazione.
Per evitare condizioni di concorrenza, il backend deve bloccare la riga della `disponibilita` interessata (ad esempio tramite `SELECT ... FOR UPDATE`) prima di verificare l'assenza di altre prenotazioni occupanti e inserire la nuova richiesta. Un indice su `(id_disponibilita, stato)` supporta la ricerca.

Medico effettivo

Il medico previsto è determinato dalla disponibilità; `id_medico_effettivo` identifica il medico assegnato all'esecuzione della prestazione. Dopo il completamento della visita identifica il medico che l'ha effettivamente eseguita.

Per una prenotazione `EFFETTUATA` il medico effettivo deve essere valorizzato e deve risultare abilitato alla prestazione.

Se differisce dal medico previsto, la differenza rappresenta una sostituzione.

9. Tabella PAGAMENTO

Campo	Tipo	NULL	Vincoli
`id_pagamento`	`INT UNSIGNED`	NO	PK, AUTO_INCREMENT
`id_prenotazione`	`INT UNSIGNED`	NO	FK
`registrato_da`	`INT UNSIGNED`	SÌ	FK
`modalita`	`VARCHAR(20)`	NO	CHECK
`importo`	`DECIMAL(10,2)`	NO	CHECK > 0
`stato`	`VARCHAR(20)`	NO	DEFAULT `IN_ATTESA` , CHECK
`provider_payment_id`	`VARCHAR(255)`	SÌ	
`creato_il`	`DATETIME`	NO	
`pagato_il`	`DATETIME`	SÌ	

Le modalità ammesse sono:

```
ONLINE
DESK_POS
DESK_CONTANTI
```

Gli stati sono:

`IN_ATTESA`
`PAGATO`
`FALLITO`

Per i pagamenti `ONLINE`, `registrato_da` deve essere assente. Per `DESK_POS` e `DESK_CONTANTI` deve invece essere valorizzato.

Se lo stato è `PAGATO`, `pagato_il` deve essere valorizzato; negli altri stati rimane assente.

Una prenotazione può conservare più tentativi di pagamento, ma deve esistere al massimo un tentativo con stato `PAGATO`. Il controllo viene effettuato dal backend, supportato dall'indice `(id_prenotazione, stato)`.

Il pagamento può essere avviato soltanto per una prenotazione confermata. Nei pagamenti desk l'utente registrante deve avere ruolo `OPERATORE_DESK`, corrispondente all'Operatore amministrativo / Segreteria.

10. Tabella REFERTO

Campo	Tipo	NULL	Vincoli
`id_referto`	`INT UNSIGNED`	NO	PK, AUTO_INCREMENT
`id_prenotazione`	`INT UNSIGNED`	NO	FK, UNIQUE
`id_medico_caricamento`	`INT UNSIGNED`	NO	FK
`nome_file`	`VARCHAR(255)`	NO	
`percorso_file`	`VARCHAR(500)`	NO	
`caricato_il`	`DATETIME`	NO	

Il vincolo `UNIQUE(id_prenotazione)` garantisce che una prenotazione possa avere al massimo un referto.

Il database conserva i metadati e il percorso del PDF; il documento vero e proprio è memorizzato in un'area privata gestita dal backend.

Il backend verifica che:

- la prenotazione sia `EFFETTUATA` ;
- `id_medico_caricamento` coincida con `id_medico_effettivo` .

11. Tabella RIMBORSO

Campo	Tipo	NULL	Vincoli
`id_rimborso`	`INT UNSIGNED`	NO	PK, AUTO_INCREMENT
`id_pagamento`	`INT UNSIGNED`	NO	FK, UNIQUE
`verificato_da`	`INT UNSIGNED`	SÌ	FK
`stato`	`VARCHAR(20)`	NO	DEFAULT `RICHIESTO` , CHECK
`importo`	`DECIMAL(10,2)`	NO	CHECK > 0
`richiesto_il`	`DATETIME`	NO	
`verificato_il`	`DATETIME`	SÌ	
`motivazione`	`VARCHAR(500)`	NO	
`provider_refund_id`	`VARCHAR(255)`	SÌ	
`rimborsato_il`	`DATETIME`	SÌ	

Gli stati ammessi sono:

```
RICHIESTO
APPROVATO
RESPINTO
IN_ELABORAZIONE
RIMBORSATO
FALLITO
```

Il vincolo `UNIQUE(id_pagamento)` consente al massimo un rimborso per pagamento.

`verificato_da` e `verificato_il` devono essere entrambi valorizzati oppure entrambi assenti.

Quando lo stato è `RIMBORSATO` , `rimborsato_il` deve essere valorizzato.

La procedura effettivamente realizzata consente al paziente di presentare una richiesta motivata per una prenotazione `NON_EROGATA` con pagamento online riuscito. Il rimborso è totale; la segreteria verifica la richiesta e la approva o la respinge secondo le condizioni applicative.

L'annullamento ordinario di una prenotazione confermata con pagamento riuscito o in attesa è bloccato; non è quindi disponibile il percorso di annullamento seguito dalla valutazione del rimborso. L'approvazione è una decisione amministrativa e non esegue la restituzione del denaro.

Gli stati `IN_ELABORAZIONE`, `RIMBORSATO` e `FALLITO`, insieme ai campi del provider e alla data di rimborso, sono previsti dallo schema; la loro presenza non implica che il flusso di restituzione sia implementato.

12. Indici

Oltre agli indici creati implicitamente da PK e `UNIQUE`, il modello prevede:

```
CREATE INDEX idx_disponibilita_medico_periodo
ON disponibilita (id_medico, data_ora_inizio, data_ora_fine);

CREATE INDEX idx_disponibilita_stanza_periodo
ON disponibilita (id_stanza, data_ora_inizio, data_ora_fine);

CREATE INDEX idx_prenotazione_disponibilita_stato
ON prenotazione (id_disponibilita, stato);

CREATE INDEX idx_pagamento_prenotazione_stato
ON pagamento (id_prenotazione, stato);
```

Gli indici supportano le verifiche più frequenti demandate ai service, in particolare disponibilità temporali, occupazione degli slot e tentativi di pagamento.

13. Utilizzo nel backend Java

L'accesso a MySQL avviene mediante JDBC ed è concentrato nel livello repository:

```
HTTP / Routing
      ↓
Controller
      ↓
Service
      ↓
Repository
      ↓
JDBC
      ↓
MySQL
```

I repository sono responsabili delle operazioni SQL e della conversione tra righe del database e oggetti Java.

I service applicano le regole di business che non possono o non conviene esprimere direttamente mediante vincoli relazionali.

Questa separazione consente di mantenere distinti:

- gestione HTTP;
- logica applicativa;
- persistenza;
- vincoli strutturali del database.